

Chahinez Benaicha

LAB 4

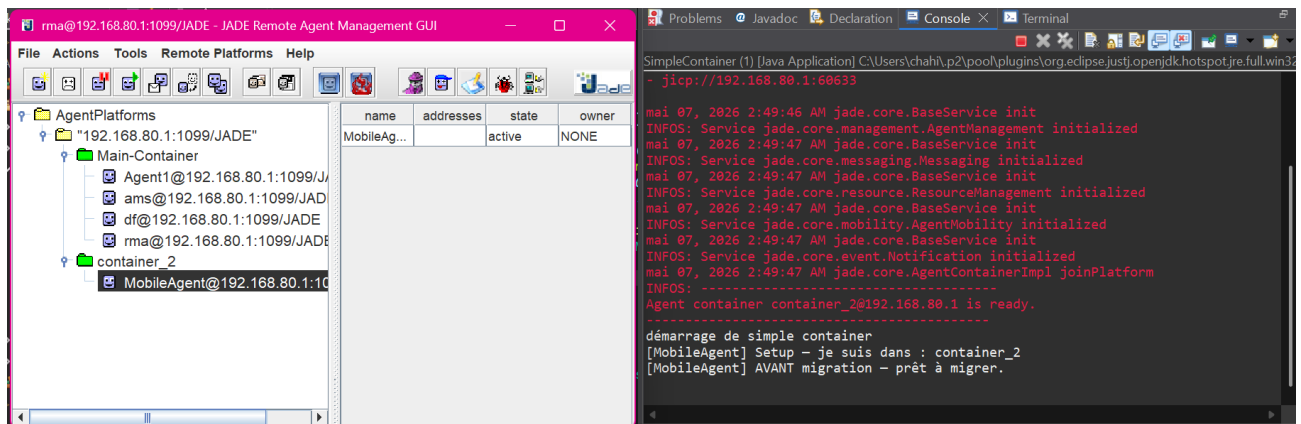
RAPPORT

Behaviours in JADE

Exercise 1: Agent migration (mobile agent)

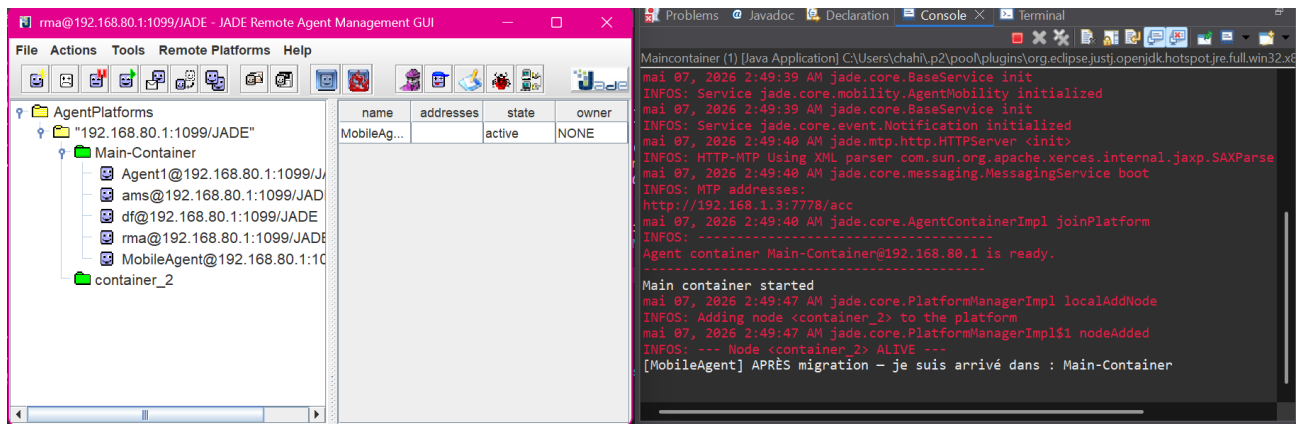
- Created an agent on a secondary container (container_2).
- Used the JADE GUI (RMA) to migrate the agent to the Main-Container.
- The agent displayed a message:

Before migration & During migration :



```
démarrage de simple container
[MobileAgent] Setup - je suis dans : container_2
[MobileAgent] AVANT migration - prêt à migrer.
[MobileAgent] PENDANT migration - je quitte : container_2
```

After migration



- This demonstrates JADE's support for mobile agents that can move between containers while preserving their state.

Exercise 2: Primitive behaviours

- Created an agent that runs several simple behaviours, each tested individually:
- Behaviour** : loops until done() equals true.

```
addBehaviour(new Behaviour() {  
    private int count = 0;  
  
    @Override  
    public void action() {  
        count++;  
        System.out.println("[Behaviour] Exécution #" + count);  
    }  
  
    @Override  
    public boolean done() {  
        return count >= 3; // s'arrête après 3 fois  
    }  
});
```

```
Démarrage de l'agent : MonAgent@192.168.80.1:1099/JADE  
[Behaviour] Exécution #1  
[Behaviour] Exécution #2  
[Behaviour] Exécution #3
```

- OneShotBehaviour** : executes once and terminates.

```
addBehaviour(new OneShotBehaviour() {  
    @Override  
    public void action() {  
        System.out.println("[OneShotBehaviour] Exécuté une seule fois.");  
    }  
});
```

```
Démarrage de l'agent : MonAgent@192.168.80.1:1099/JADE  
[OneShotBehaviour] Exécuté une seule fois.
```

- CyclicBehaviour** : executes repeatedly until agent kills it.

```
addBehaviour(new CyclicBehaviour() {  
    private int count = 0;  
  
    @Override  
    public void action() {  
        count++;  
        System.out.println("[CyclicBehaviour] Cycle #" + count);  
        if (count >= 3) removeBehaviour(this); // arrêt manuel  
    }  
});
```

```
Démarrage de l'agent : MonAgent@192.168.80.1:1099/JADE  
[CyclicBehaviour] Cycle #1  
[CyclicBehaviour] Cycle #2  
[CyclicBehaviour] Cycle #3
```

- WakerBehaviour** : executes once after a specified delay.

```
addBehaviour(new WakerBehaviour(this, 6000) {  
    @Override  
    protected void onWake() {  
        System.out.println("[WakerBehaviour] Réveil après 6s → agent terminé.");  
        doDelete();  
    }  
});
```

```
Démarrage de l'agent : MonAgent@192.168.80.1:1099/JADE  
[WakerBehaviour] Réveil après 6s → agent terminé.  
Agent terminé.
```

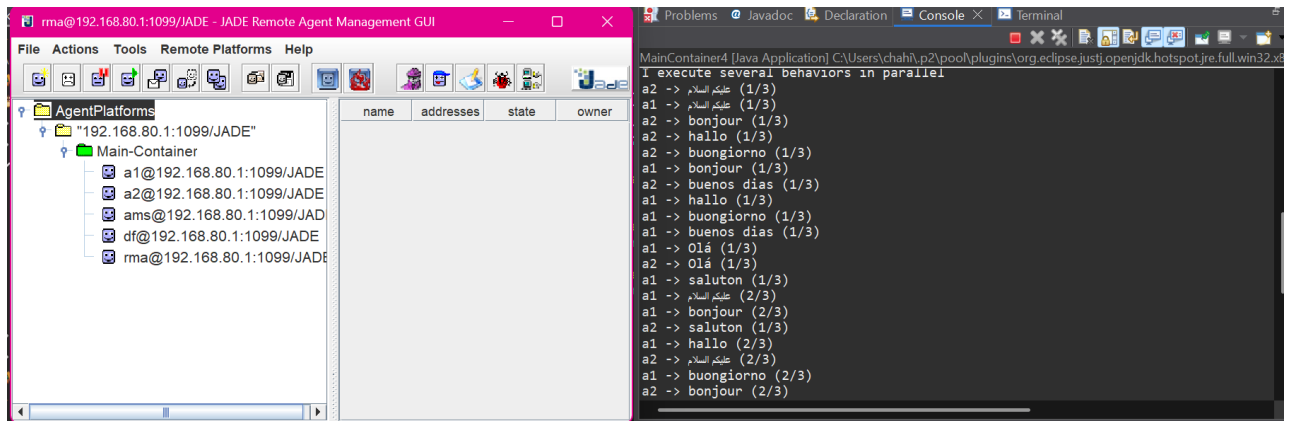
- TickerBehaviour** : executes periodically at fixed intervals.

```
addBehaviour(new TickerBehaviour(this, 2000) {  
    @Override  
    protected void onTick() {  
        counter++;  
        System.out.println("[TickerBehaviour] Tick #" + counter);  
        if (counter >= 3) removeBehaviour(this); // arrêt manuel  
    }  
});
```

```
Démarrage de l'agent : MonAgent@192.168.80.1:1099/JADE  
[TickerBehaviour] Tick #1  
[TickerBehaviour] Tick #2  
[TickerBehaviour] Tick #3
```

Exercise 3: Composite behaviour

- Created two agents (a1 and a2).
- Each agent owns multiple behaviours that execute **in parallel**. each behaviour is activated **3 times**, and displayed greetings in several languages.
- Composite behaviours:**
 - ParallelBehaviour**: executes multiple child behaviours concurrently.
 - SequentialBehaviour**: executes child behaviours one after another.



Conclusion

This lab provided a comprehensive understanding of behaviour-based agent programming in JADE.

Primitive behaviours allow simple tasks, while composite behaviours enable sophisticated concurrency patterns.